

OFT Factory Assessment *Blueprint Finance*

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE

Assurance Assessment

DATE OF ENGAGEMENT

Aug 24, 2026 - Aug 25, 2026

SOURCE CODE

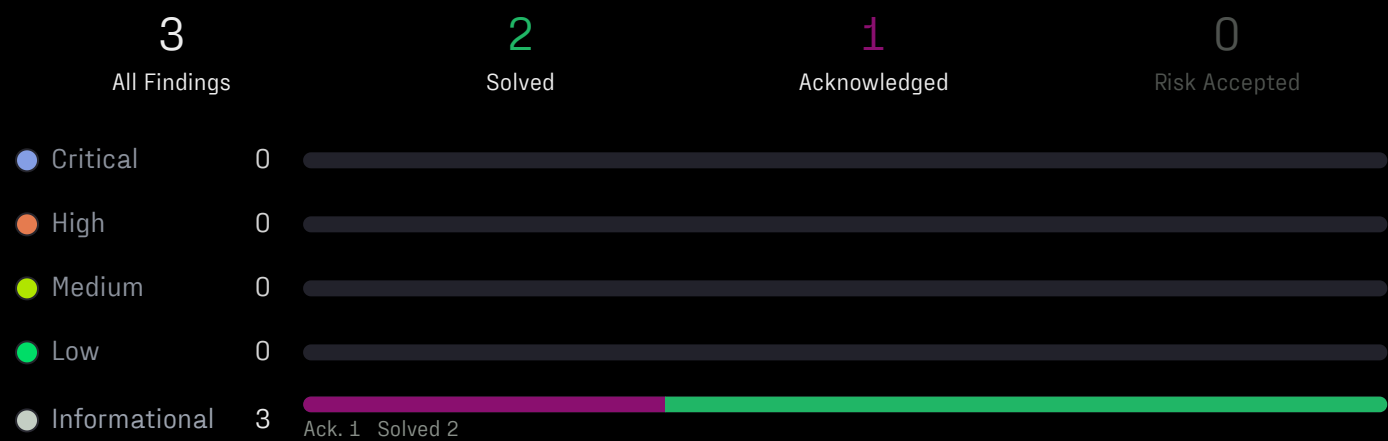
 ASSETcx

 /ASSETcx-b6ae62b0d1b393ec2d9d85b0e237ce977

ASSESSED COMMIT ID

 b6ae62b

SEVERITY BREAKDOWN



INTRODUCTION

Blueprint Finance engaged Halborn to perform a security assessment of their smart contracts on August 24th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The Blueprint Finance codebase in scope consists of an upgradeable ERC20 with EIP-2612 permit and native LayerZero omnichain transfers, paired with a lockbox style OFT adapter that custodies the AssetCX token for cross chain movement.

ASSESSMENT SUMMARY

Halborn was allocated 1 day for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, **Halborn** did not identify any significant security risks. However, some enhancements were recommended to improve code clarity, consistency, or maintainability. These were partially addressed by the **Blueprint Finance** team. The main recommendations were:

- **Override OwnableUpgradeable._transferOwnership() in AssetCX0FT and AssetCX0FTAdapter to also set the LayerZero endpoint delegate to the new owner within the same call, keeping the delegate aligned with the owner in one transaction.**
- **In AssetCX0FT.initialize(), derive the EIP-712 domain name from _name so it always equals the ERC20 token name, keeping the reconstructed domain separator consistent for EIP-2612 tooling.**
- **Update the AssetCX0FTAdapter NatSpec header to state that it wraps the AssetCX ERC20, matching the constructor _token parameter.**

I TEST APPROACH AND METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (**Foundry**).
- Fork testing against main networks (**Foundry**).
- Static security analysis of scoped contracts, and imported functions (**Slither**).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Medium (AC:M)	0.67
	High (AC:H)	0.33
	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = \max(m_I) + \frac{\sum m_I - \max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORY
(a) Repository:  ASSETcx
(b) Assessed Commit ID:  b6ae62b0d1b393ec2d9d85b0e237ce97754343eb
(c) Items in scope:

✓  src 2 files

- <> AssetCXOFT.sol Solidity
- <> AssetCXOFTAdapter.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

FILE

(a) Submitted File:  /ASSETcx-b6ae62b0d1b393ec2d9d85b0e237ce97754343eb/src/AssetCXOFT.sol

(b) Items in scope:

✓  ASSETcx-b6ae62b0d1b393ec2d9d85b0e237ce97754343eb/src 2 files

- <> AssetCXOFT.sol Solidity
- <> AssetCXOFTAdapter.sol Solidity

REMEDIATION COMMIT ID:

✦ e80d9a8fb81e9f8b5c85120adc4b1ca4ad93a45d

✦ 683f33790f5f2d6f4e93913a33bdd5b814da2334

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Temporary delegate and owner mismatch after ownership transfer	• Informational	0.9	ACKNOWLEDGED 09/03/2026
HAL-002	Permit signature verification failures in off-chain tooling due to EIP-712 domain name diverging from token name	• Informational	0.6	SOLVED 09/03/2026
HAL-003	Misleading adapter documentation due to incorrect wrapped token reference	• Informational	0.3	SOLVED 09/03/2026

Findings & Tech Details

HAL-001

ACKNOWLEDGED

Temporary delegate and owner mismatch after ownership transfer

• Informational · 0.9 A0:S/AC:L/AX:L/R:P/S:U/C:N/A:M/I:H/D:N/Y:N

DESCRIPTION

The `AssetCX0FT` and `AssetCX0FTAdapter` contracts set the LayerZero endpoint delegate and the `Ownable` owner to the same address at initialization, where the owner controls contract configuration and upgrades while the endpoint delegate controls OApp settings such as messaging libraries and verification.

However, both contracts inherit the single step `OwnableUpgradeable`, whose `transferOwnership()` updates only the owner and leaves the endpoint delegate pointing at the previous owner.

After an ownership handover, the previous owner keeps endpoint level configuration authority until the delegate is realigned, so the new and old authorities briefly diverge.

The mismatch is transient and self recoverable: the new owner can restore `delegate == owner` in a single `onlyOwner setDelegate()` call, after which the former owner can no longer reclaim the delegate. Acting on the window requires the already trusted previous owner to do so before it is corrected.

RECOMMENDATION

Override `OwnableUpgradeable._transferOwnership()` in `AssetCX0FT` and `AssetCX0FTAdapter` to also set the LayerZero endpoint delegate to the new owner within the same call, keeping the delegate aligned with the owner in one transaction.

Additionally, adopt a two step ownership handover so the incoming owner must accept the role before it takes effect.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Blueprint Finance` team made a business decision to acknowledge this finding and not alter the contracts, stating:

We choose to keep these two roles disentangled. While the owner does not need to equal the delegate, in our current business logic it always does. However, we intentionally do not want to disallow these two roles from being held by different accounts, as this flexibility may be needed in the future. Additionally, we do not consider a two-step ownership transfer necessary for these contracts at this point, and acknowledge the associated risk."

HAL-002

SOLVED

Permit signature verification failures in off-chain tooling due to EIP-712 domain name diverging from token name

• Informational · 0.6 AO:A/AC:H/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `AssetCX0FT.initialize()` function sets the ERC20 token name from the `_name` argument and, independently, sets the EIP-712 permit domain name from the `eip712Name` argument, so the two values are supplied separately at initialization.

However, nothing constrains `eip712Name` to equal `_name`. Off-chain tooling that follows the EIP-2612 convention reconstructs the permit domain separator from the token's `name()`, and when the two diverge that reconstruction yields a domain separator the token does not use.

Permit signatures built by such tooling then fail verification, so gasless approvals stop working for those integrators. The on-chain `permit()` path and consumers of `DOMAIN_SEPARATOR()` or the ERC-5267 `eip712Domain()` getter read the contract's own domain and remain unaffected, and the divergence only arises if a deployer passes `eip712Name != _name`.

RECOMMENDATION

In `AssetCX0FT.initialize()`, derive the EIP-712 domain name from `_name` so it always equals the ERC20 token name, keeping the reconstructed domain separator consistent for EIP-2612 tooling.

REMEDIATION COMMENT

Addressed in: [🔗 e80d9a8fb81e9f8b5c85120adc4b1ca4ad93a45d](#)

SOLVED: The **Blueprint Finance** team solved this finding in the specified commit by removing the independent `eip712Name` parameter from `initialize()` and initializing the permit domain with `__ERC20Permit_init(_name)`, so the EIP-712 domain name always equals the ERC20 token name.

HAL-003

SOLVED

Misleading adapter documentation due to incorrect wrapped token reference

• Informational · 0.3 A0:S/AC:M/AX:L/R:P/S:U/C:N/A:N/I:M/D:N/Y:N

DESCRIPTION

The `AssetCX0FTAdapter` contract is a LayerZero OFT adapter that locks the AssetCX ERC20 on its home chain and pairs with `AssetCX0FT` mint and burn contracts on the side chains, giving the asset a single canonical accounting path across the mesh. The locked balance stays equal to the outstanding remote supply because `AssetCX0FTAdapter` credits recipients only through the peer gated `LzReceive` path, which admits one peer per source endpoint.

However, the contract NatSpec header describes it as wrapping `AssetCX0FT`, while the constructor `_token` parameter is documented as the AssetCX ERC20 that the adapter actually locks. The header names the wrong underlying token.

The inconsistency is a documentation defect that can mislead integrators about which token the adapter wraps, without affecting on chain behavior.

RECOMMENDATION

Update the `AssetCX0FTAdapter` NatSpec header to state that it wraps the AssetCX ERC20, matching the constructor `_token` parameter.

Additionally, keep the deployment peered so each underlying asset has a single lock and mint accounting path across the mesh.

REMEDIATION COMMENT

Addressed in: [683f33790f5f2d6f4e93913a33bdd5b814da2334](#)

SOLVED: The `Blueprint Finance` team solved this finding in the specified commit by updating the `AssetCX0FTAdapter` NatSpec header to state it wraps `AssetCX` (ERC20) instead of `AssetCX0FT`, matching the constructor `_token` parameter.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.

